CPE 314 - Software Design

University of Cebu

Computer Engineering Department

Campus Marketplace with Reviews

Project Design Document

Table 1: Group information

Group Name / No.	Group 1 / Team 01 - BSCPE 3B - EDP 69323
Members	John Benedict Alaiza
	Acexion B. Mahigugmaon
	Glyde V. Arreglado
	Tristan Dominic Urot
	Joshua Brian S. Enovero
	Emanuelle De Paz
Instructor	Richyll Son
Term	First Semester, S.Y. 2026-2027
Document Version / Date	V 0.3 / 13 November 2026

Table of Contents

1. Introduction & Project Overview 1

2. Preliminary ER Sketch & Team Roles 1

3. System Architecture & Object-Oriented Design 2

- Class hierarchy . . . 2
- Class responsibilities . . . 3
- Condition rules . . . 3
- Supporting components . . . 4
- Collections and recursion . . . 5
- Application screens and state . . . 5
- Sign in, registration and password recovery . . . 5
- Marketplace screens and ownership . . . 6
- Listing types and sample records . . . 7
- Categories and listing rules . . . 7

4. Data Model & Database Design 8

- Finalized entity relationships . . . 8
- Categories and historical sales . . . 10
- Listing condition details . . . 11
- Whose ID is it? . . . 12
- Account fields and database upgrades . . . 12
- Relationships and storage rules . . . 13

5. API Integration Design 16

- Catalog request and response . . . 16
- Mapping and listing-form use . . . 16
- Errors and API verification . . . 16

6. Concurrency Design 17

- Worker and controller state . . . 17
- Background JSON import . . . 17
- Automatic updates between windows . . . 17
- Responsiveness and shutdown . . . 17

7. Data Mining & Visualization Design 18

- Weekly sales frequency in Java . . . 18
- Category averages . . . 18
- Charts and why they fit . . . 18
- Checking that the analysis changes . . . 18

Appendix - References and reproduction 19

- Account setup and recovery . . . 19
- Verification . . . 19
- Limitations and requirement mapping . . . 19
- Program process map . . . 20
- Complete Java type inventory . . . 21

Revision History

Table 2: Document revisions

Version	Date	Author	Changes
0.1	23 Oct 2026	Group 1	Initial project overview, ER sketch and responsibilities.
0.2	8 Nov 2026	Group 1	Documented the listing classes, named the IDs and clarified the table connections.
0.3	13 Nov 2026	Group 1	Registration, MVC, Java analysis, listing dates/status, safer detail handling and matching diagrams/tests.

1. Introduction & Project Overview

Campus Marketplace with Reviews lets students, faculty and school groups buy and sell textbooks, electronics and furniture. Members search categories, join waitlists, review listings and compare category prices and ratings. Each item type calculates its own condition grade.

The JavaFX app stores records in SQLite and imports JSON sales history. Members register or use the initial local accounts, then sign in to Marketplace, Sell an item, Sales history and Analytics. Buyer-seller messages and offers are out of scope; the app has no message/offer table or chat screen.

2. Preliminary ER Sketch & Team Roles

Each box is a table. Lines join primary keys (PK) to foreign keys (FK). The fork marks the many side of 1:M. Section 4 shows all fields, including login and recovery data.

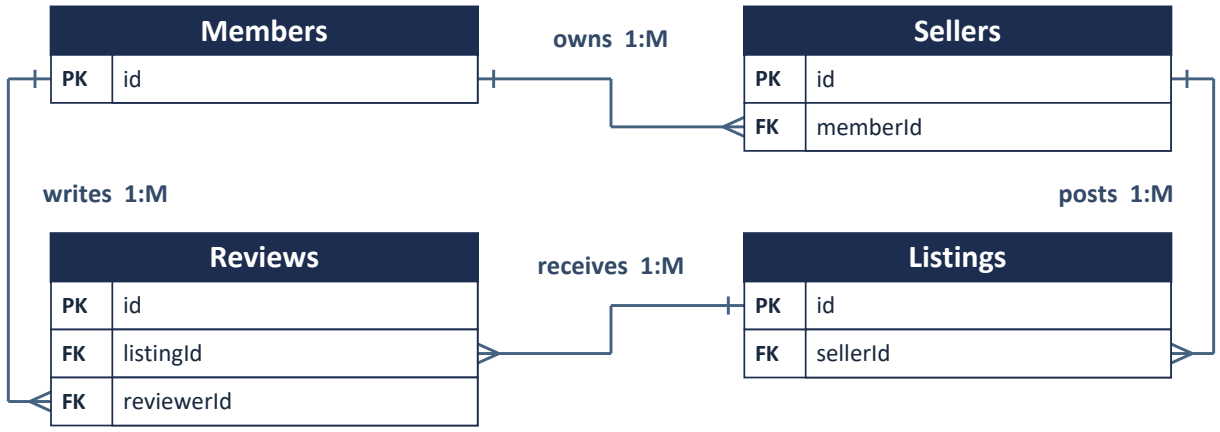


Figure 1: Preliminary ownership and review relationships

Table 3: Team responsibilities

Member and role	Responsibility
John Benedict Alaiza - Backend Developer (Listing Models)	Listing classes, condition rules and validation.
Acexion B. Mahigugmaon - Project Leader / Integration	Connect the components, maintain documents and coordinate fixes.
Glyde V. Arreglado - Database Admin	SQLite tables, saved records and summary queries.
Tristan Dominic Urot - UI/GUI Developer	JavaFX controls, event handlers and charts.
Joshua Brian S. Enovero - Backend & API Developer	Catalog requests, JSON parsing and import.
Emanuelle De Paz - QA & Testing	Test cases, expected calculations and error checks.

3. System Architecture & Object-Oriented Design

Models hold item rules. Views show controls, controllers coordinate actions, services apply rules and DAOs run feature SQL. Database opens SQLite and initializes records. App connects these parts when the program starts.

Class hierarchy

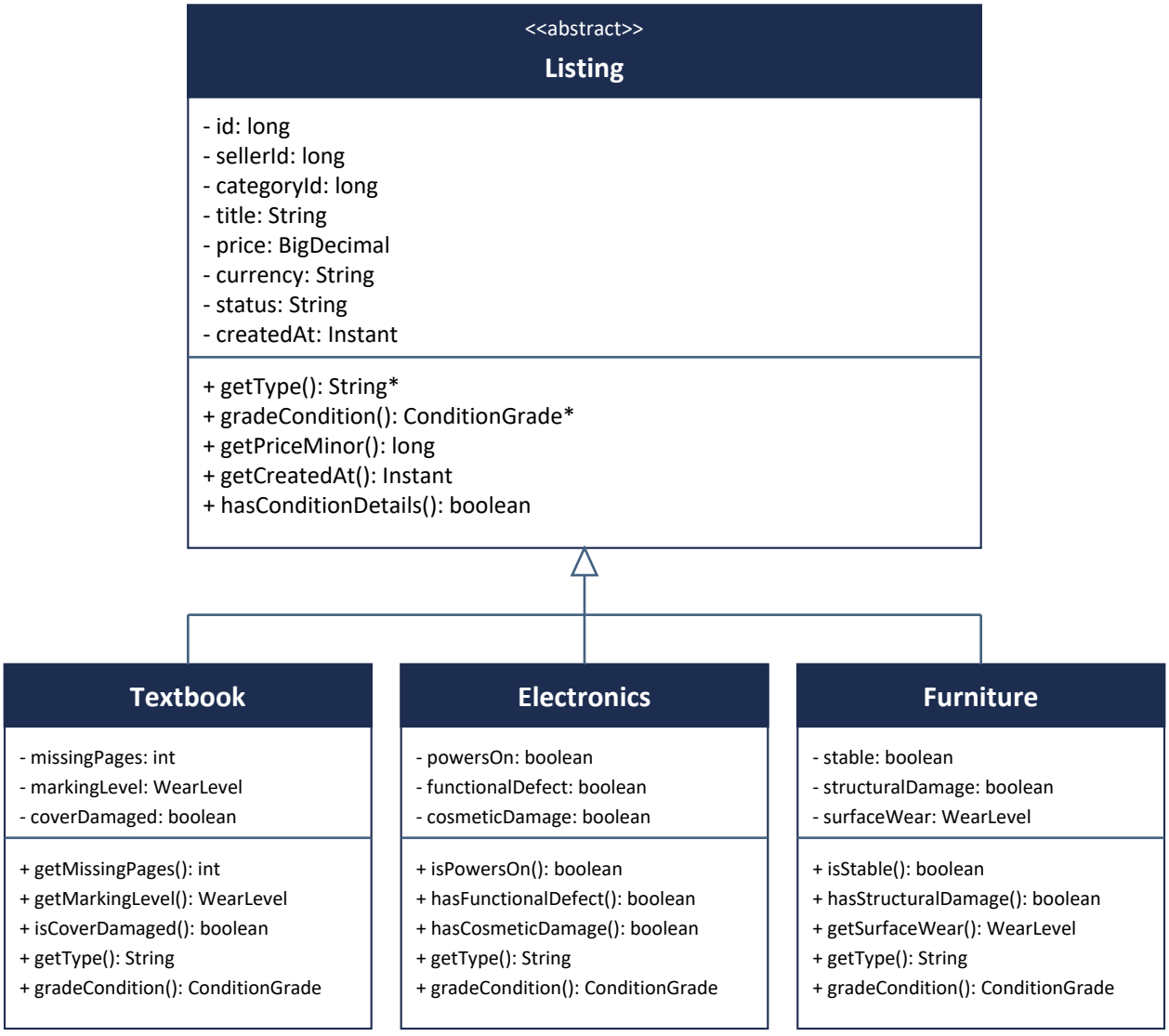


Figure 2: Listing class hierarchy and type-specific condition rules

The hollow triangle points to the parent class. A minus sign marks a private field and a plus sign marks a public method. An asterisk marks an abstract method. Each subclass inherits the shared listing data and public getters, then supplies its own item type and condition grade.

Class responsibilities

The shared Listing fields are ID, seller/category IDs, title, price, currency, status and creation time. The abstract methods getType() and gradeCondition() are implemented by each subclass. A List<Listing> can hold all three types. Calling gradeCondition() through Listing runs the actual object's method; this is runtime polymorphism.

Table 4: Model responsibilities and Single Responsibility Principle

Class	Responsibility and reason to change
Listing	Shared listing values and validation. Changes when common listing rules change.
Textbook	Textbook evidence and condition grade. Changes when textbook condition rules change.
Electronics	Electronics evidence and condition grade. Changes when electronics condition rules change.
Furniture	Furniture evidence and condition grade. Changes when furniture condition rules change.
ConditionGrade / WearLevel	Allowed grade and wear values. Changes when these allowed values change.
Member / Category	Member identity and category parent data. Changes when these records change.

Keeping item rules separate from SQL and controls follows the Single Responsibility Principle. A changed grading rule belongs in its item class.

Condition rules

• Textbook: missing pages = POOR; heavy markings = FAIR; light markings or cover damage = GOOD; otherwise LIKE_NEW.

• Electronics: no power or a functional defect = POOR; cosmetic damage = GOOD; otherwise LIKE_NEW.

• Furniture: unstable or structurally damaged = POOR; heavy wear = FAIR; light wear = GOOD; otherwise LIKE_NEW.

Supporting components

Table 5: Application layers and responsibilities

Layer	Responsibility and reason to change
App	Creates the database, DAOs, services, controllers and views. Owns the shared worker and shutdown.
Views in ui	Draw controls, bind state, show dialogs and forward actions. Contain no SQL, API calls or worker creation.
Controllers	Hold observable state, obtain the active member from UserSession and request background work.
Services	Apply account, listing, review and analysis rules. Convert data-access failures before they reach the screen.
DAOs	Execute prepared SQL and return records. Recheck storage-sensitive rules in write transactions.
Database / AccountSetup	Open connections, create tables, seed initial records and migrate account fields without erasing saved data.
ProductCatalogClient / ProductCatalogParser	Download the catalog and validate its JSON in separate classes.
CatalogProduct / CatalogResponse	Carry the parsed product subset and HTTP information in dto.
CatalogMapper / CatalogService	Select supported comparable products and prepare reference information.
SalesHistoryImporter / SalesAnalysis	Stream and validate JSON imports; group saved timestamps and count weekly activity in Java.

The action path is View -> Controller -> Service -> DAO -> SQLite. Task results update controller properties on the JavaFX thread; bound views display them. Models contain no GUI or SQL. The Appendix lists every top-level Java type and the process map.

Collections and recursion

Waitlists use Queue with ArrayDeque: offer() adds at the tail and poll() reads the first buyer. Stored joiningOrder values rebuild FIFO order. InterestDao assigns MAX(joiningOrder)+1 and inserts in one atomic INSERT ... SELECT, with eligibility rechecked in that same statement. SQLite serializes writers; ties do not receive the same position. A Map stores each member's Stack of recently viewed IDs. push() adds the latest view; pop() reads a copy, preserving session history. At most 20 unique views are kept per member.

collectCategoryIds() adds a category ID and recursively visits each child. A category without children is the base case. Validation rejects missing parents, duplicate IDs and cycles before traversal. Search includes the selected category and all descendants.

Application screens and state

MarketplaceView is the application shell. LoginView, RegistrationView and PasswordResetView contain account forms. ListingTableView and ListingCards display the same observable listing list. ListingFormView handles posting inputs, ReviewsView contains review/waitlist controls, ImportView displays import state and ChartsView renders the returned results.

UserSession holds the active Member and a generation number. AccountController, ListingController, ReviewController, WaitlistController and AnalyticsController coordinate their features. FxTaskRunner exposes busy, status and error properties. RefreshController decides whether to poll; FxRefreshSchedule supplies the timer. These per-window values are in memory, not new database columns.

InitialAccount, Registration, ImportResult, CategorySummary, ListingForRepair, WeeklySales and AnalyticsSnapshot carry setup/action results. Review and Seller hold domain values. ItemArtwork draws category illustrations, and UiTools provides presentation helpers. Artwork is not a separate database entity.

Sign in, registration and password recovery

Registration requires a display name of 1-80 characters, a username of 3-30 letters/digits/underscores and a password of 8-128 characters. MemberDao strips and lowercases the username and rejects duplicates through the unique index. It stores salted PBKDF2 hashes and returns a recovery code once. The member saves that code and then signs in; registration does not automatically sign in.

A new database creates Person A, Person B and Person C with usernames person_a, person_b and person_c. Each receives a random password and recovery code. A local startup dialog shows newly issued credentials once. They are not published here or stored as plaintext in SQLite. Existing current accounts retain their credentials when reopened.

Login verifies the password hash and sets UserSession. Forgot password verifies the saved recovery code and atomically replaces the password hash and recovery-code hash. The screen displays a replacement code to save; the old code cannot be reused. Failed resets preserve the previous credentials. This is local recovery, not email delivery.

Hashes use PBKDF2WithHmacSHA256, 210,000 iterations and 256 bits. Random salts use 16 bytes; recovery values use 18 random bytes encoded to 24 URL-safe characters. CredentialSecurity holds these settings. Controllers clear submitted password/recovery character arrays after use.

Marketplace screens and ownership

The app opens maximized with navigation across the top. Selecting a card or list row opens a closable details panel on the right. Cards and List show the same records, dates and statuses. Read reviews opens a right-positioned popup with names, stars and comments; Review listing opens a form on that side. Smaller panels scroll. View details loads the waitlist and recent views.

Each member may buy or sell. Posting uses the member's seller profile, created when needed. My listings filters to that owner and can show All statuses, Active, Sold or Removed. Mark as sold and Remove listing require confirmation and recheck ownership in SQL. Records, condition rows, reviews and waitlists are retained. Public search and current-listing means include ACTIVE items only.

Join waitlist accepts an existing member different from the seller and saves joiningOrder. Repeated joins are rejected. Reviews require membership in that listing's waitlist, another owner and no earlier review by the same member/item. Ratings are 1-5; optional comments have at most 1,000 characters. The item must remain ACTIVE. Waitlist membership demonstrates interest, not proof of payment.

Restore missing details lets an owner choose a listing and the condition actually observed. No condition is selected automatically. Saving rechecks owner, type and missing subtype rows; it preserves the listing ID, common fields, reviews and waitlist. Cancel changes nothing. Conflicting existing details require maintainer review.

Listing types and sample records

Listing defines the shared item data. The application accepts three concrete types: Textbook, Electronics and Furniture. Database seeds one example of each when it creates the demo data. The user can post additional items of any supported type.

Table 6: Listings supplied with the prototype

Listing and type	Category and price	Saved observations and result
Java textbook (demo) / TEXTBOOK	Textbooks / PHP 350.00	No missing pages, light markings, intact cover. Grade: GOOD.
Gaming laptop (demo) / ELECTRONICS	Gaming Laptops / PHP 18,000.00	Powers on, no functional defect, cosmetic damage. Grade: GOOD.
Study chair (demo) / FURNITURE	Furniture / PHP 650.00	Stable, no structural damage, light surface wear. Grade: GOOD.

Categories and listing rules

All finds is the root category. Its direct children are Textbooks, Electronics and Furniture. Under Electronics, Computers contains Laptops, which contains Gaming Laptops. Searching a parent category includes items in its descendants.

Each listing requires a seller profile, category, nonblank title, positive exact minor-unit price and three-letter uppercase currency. A post starts ACTIVE under the signed-in seller. ListingDao saves created_at as a UTC timestamp when inserting. The owner can change ACTIVE to SOLD or REMOVED; both leave public search. No payment or historical sale is created by marking an item sold.

4. Data Model & Database Design

Finalized entity relationships

The schema defines ten application tables with 50 stored fields. The overview shows all tables and their PK/FK connections. The three detail views show every attribute at a larger size; Listings repeats to connect the views. Field names match SQLite exactly. The SQL below specifies types, nullability and checks.

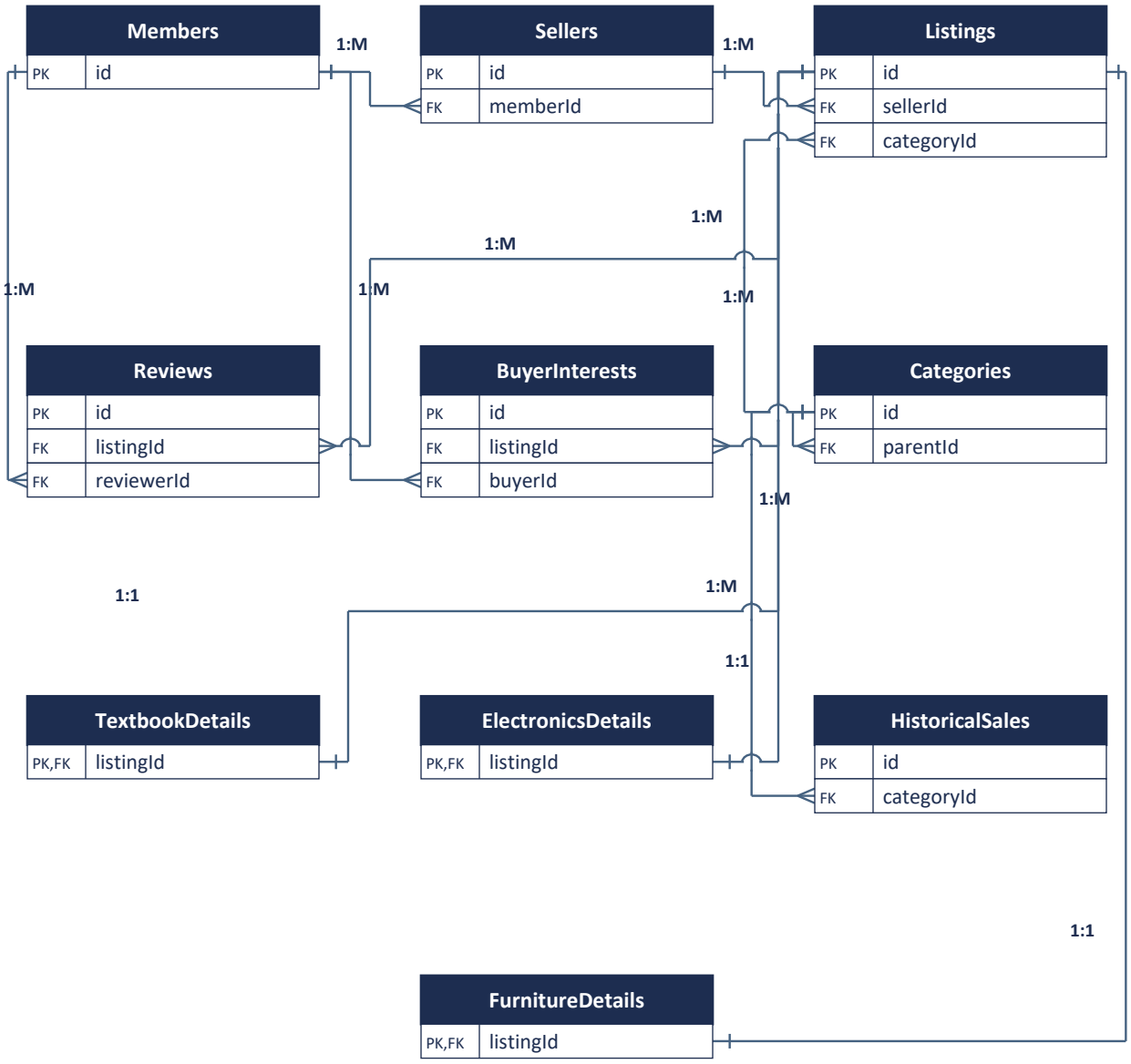


Figure 3: Ten-table overview with PK/FK connections

The overview shows keys only. The detail views and SQL show all remaining attributes. A small bridge at a crossing means the lines do not join.

Read 1:M as one parent to many children. A parent can have no children. M:N (also called M:M) means many-to-many. Members and Listings have this relationship through BuyerInterests for waitlists and through Reviews for ratings. Each connecting table has two 1:M links. Each individual listing subtype uses 1:1; only the matching subtype has a row.

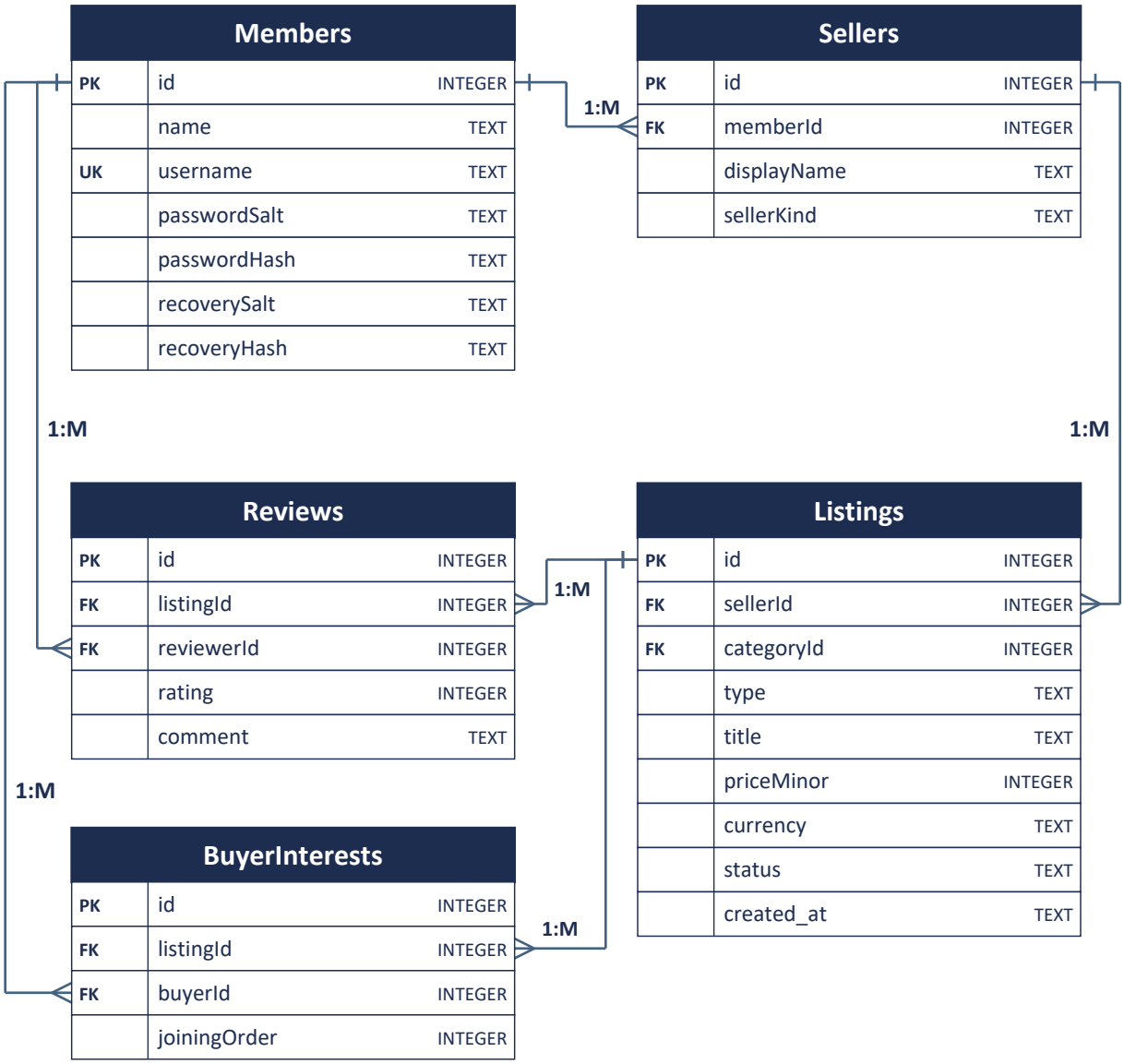


Figure 4: Members, sellers, listings, reviews and buyer interests

Categories and historical sales

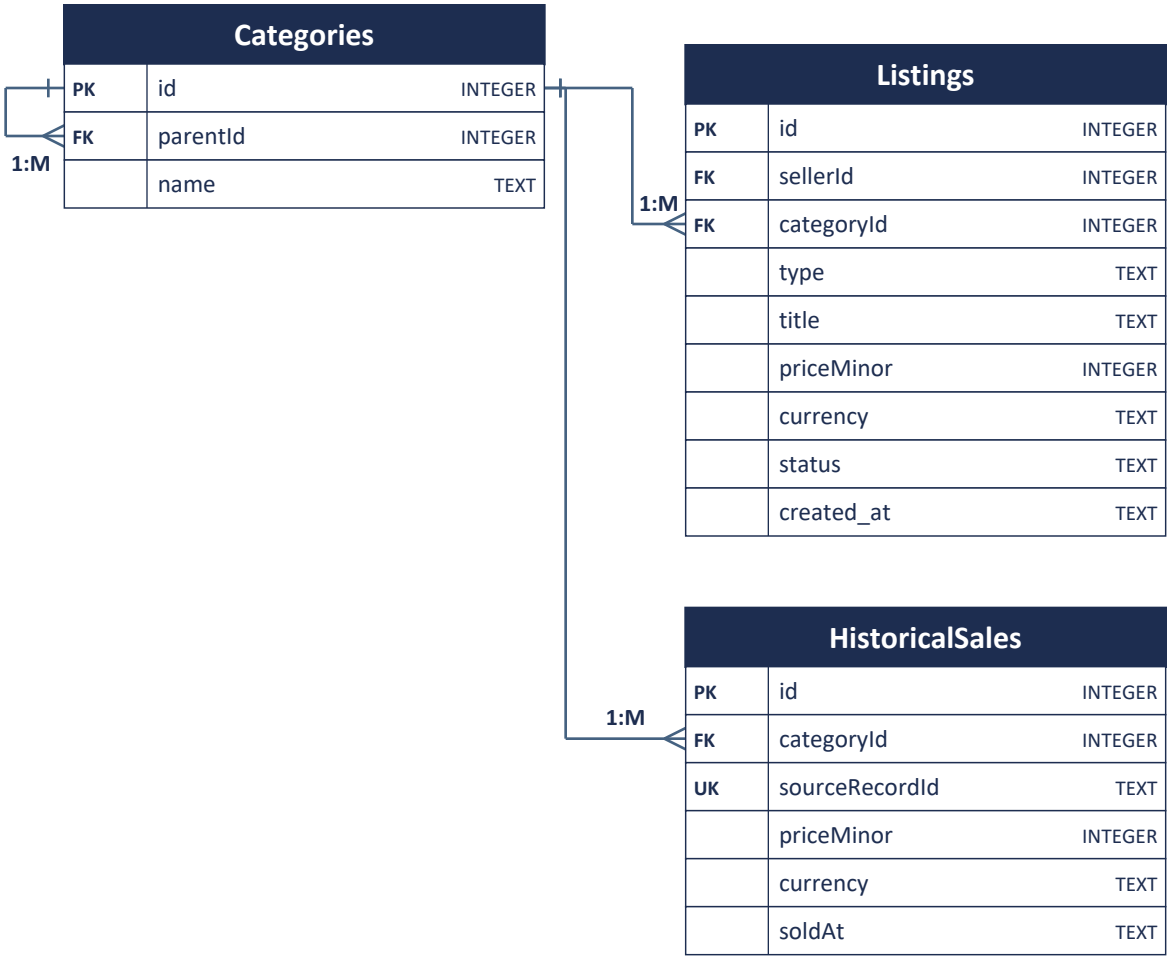


Figure 5: Categories, current listings and imported historical sales

A root category has NULL parentId. A non-root category has one parent, and a parent can have zero or more children. HistoricalSales contains imported past sales, not new marketplace listings. sourceRecordId prevents duplicate imports.

Listing condition details

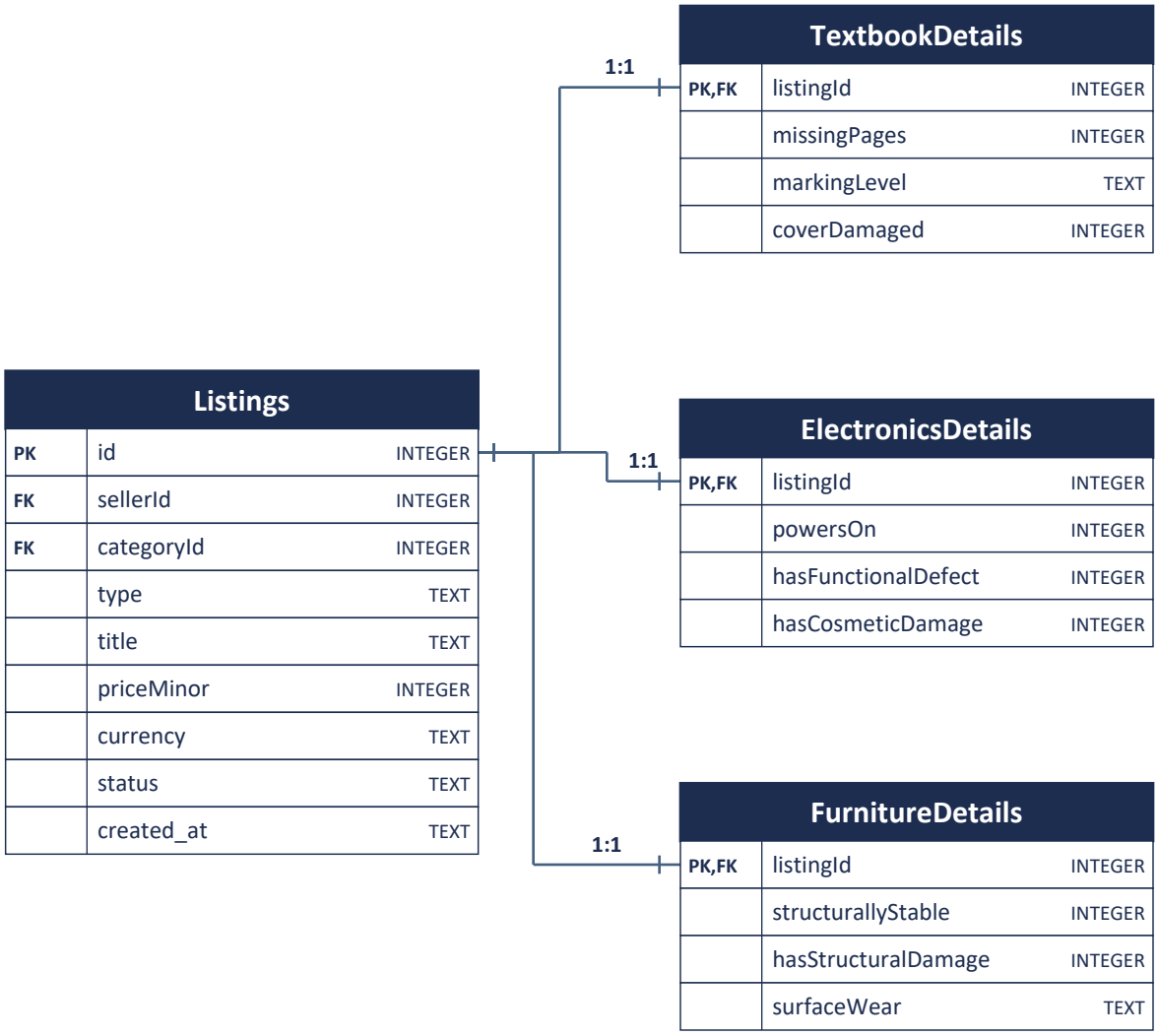


Figure 6: Shared listing fields and type-specific condition evidence

A valid listing needs exactly one matching detail row overall. It has zero or one row in each individual detail table, depending on type. A detail row's listingId is both its primary key and its foreign key. The app saves the listing and detail row in one transaction, and checks the type when reading. Foreign keys alone do not guarantee exactly one matching subtype. If details are absent or inconsistent, Listing.incomplete() returns a read-only placeholder with the saved type and common fields. Cards, tables and details show Details unavailable; no grade or boolean flag is invented and healthy items still load. Owners can restore missing rows; conflicting rows require maintainer review.

Some Java names are shorter than their SQL columns. Electronics.functionalDefect maps to hasFunctionalDefect, and cosmeticDamage maps to hasCosmeticDamage. Furniture.stable maps to structurallyStable, and structuralDamage maps to hasStructuralDamage. ListingDao converts boolean values to SQLite 0/1 and wear enums to their text names.

Whose ID is it?

A primary key (PK) identifies one row in its own table. A foreign key (FK) identifies a related row. The name id belongs to its containing table: Members.id identifies a member, Sellers.id a seller profile and Listings.id an item. Boxes use exact column names; the table below explains them.

Table 7: IDs used to connect marketplace records

SQLite field	What it identifies
Sellers.memberId	The member who owns the seller profile. Links to Members.id.
Listings.sellerId	The seller profile that posted this item. Links to Sellers.id.
Listings.categoryId	The item's category. Links to Categories.id.
Categories.parentId	The parent category. Links to Categories.id; empty for a root category.
Reviews.listingId	The listing being reviewed. Links to Listings.id.
Reviews.reviewerId	The member who wrote the review. Links to Members.id.
BuyerInterests.listingId	The listing whose waitlist the buyer joined. Links to Listings.id.
BuyerInterests.buyerId	The member who joined the waitlist. Links to Members.id.
HistoricalSales.categoryId	The category of the imported sale. Links to Categories.id.
TextbookDetails.listingId, ElectronicsDetails.listingId, FurnitureDetails.listingId	In each table, this is both PK and FK. It identifies the listing whose condition is described and links to Listings.id.

Example: Reviews.id = 7, Reviews.listingId = 12 and Reviews.reviewerId = 3 mean that member 3 wrote review 7 about listing 12.

HistoricalSales.id identifies the locally saved sale. HistoricalSales.sourceRecordId is the unique ID supplied by the JSON file to recognize a repeat import.

Account fields and database upgrades

Members.id identifies the member. username is marked UK because its unique index keeps login names distinct. passwordSalt and passwordHash verify the entered password. recoverySalt and recoveryHash verify a recovery code. Salts and hashes are saved as text. Member objects passed to the screen contain only id and name.

Database.initialize() calls AccountSetup.upgrade() to migrate versions 1-3 to version 4 in one transaction. It preserves credentials, IDs and related records; adds missing account fields, the username index and Listings.created_at; and maps old AVAILABLE/WITHDRAWN values to ACTIVE/REMOVED. PRAGMA user_version records version 4.

Listings.created_at maps to Java Listing.createdAt (Instant) and records UTC creation time on insert. Legacy rows keep NULL because their original date is unknown; the UI shows Date not recorded. Status uses ACTIVE, SOLD or REMOVED. This adds one field, not another table.

Relationships and storage rules

Table 8: Relationship cardinalities

Parent	Child	Relationship
Members	Sellers	1:M
Members	Reviews	1:M
Members	BuyerInterests	1:M
Sellers	Listings	1:M
Categories	Listings	1:M
Categories	HistoricalSales	1:M
Categories	Categories	1:M; a root has no parent
Listings	Reviews	1:M
Listings	BuyerInterests	1:M
Listings	Each detail table	1:1; only the matching subtype applies

Prices use integer minor units: PHP 350.50 is stored as priceMinor 35050 and currency PHP. Java uses BigDecimal. The app uses ACTIVE, SOLD and REMOVED status values. Required names are nonblank; review ratings range from 1 to 5. ReviewDao also rejects self-review, repeat reviews and members who have not joined the listing's waitlist inside its write transaction.

BuyerInterests has UNIQUE(listingId, buyerId) to prevent repeated interest and UNIQUE(listingId, joiningOrder) to prevent tied positions within a waitlist. Members and Listings have an M:N interest relationship implemented through BuyerInterests.

Foreign keys are enabled on every SQLite connection. PreparedStatement binds input values to SQL. Each operation opens its own connection. Saving a listing with its detail record and restoring missing details both use transactions and roll back on failure.

The following CREATE TABLE statements match program/src/main/resources/db/schema.sql. Each snippet defines one table. The username index is created during AccountSetup.upgrade().

Members

```
CREATE TABLE Members (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL CHECK (length(trim(name)) > 0),
    username TEXT,
    passwordSalt TEXT,
    passwordHash TEXT,
    recoverySalt TEXT,
    recoveryHash TEXT
);
```

AccountSetup.upgrade() also runs this index statement.

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_Members_username ON Members(username);
```

Sellers

```
CREATE TABLE Sellers (
    id INTEGER PRIMARY KEY,
    memberId INTEGER NOT NULL REFERENCES Members(id),
    displayName TEXT NOT NULL CHECK (length(trim(displayName)) > 0),
    sellerKind TEXT NOT NULL CHECK (length(trim(sellerKind)) > 0)
);
```

Categories

```
CREATE TABLE Categories (
    id INTEGER PRIMARY KEY,
    parentId INTEGER REFERENCES Categories(id),
    name TEXT NOT NULL CHECK (length(trim(name)) > 0),
    CHECK (parentId IS NULL OR parentId <> id)
);
```

Listings

```
CREATE TABLE Listings (
    id INTEGER PRIMARY KEY,
    sellerId INTEGER NOT NULL REFERENCES Sellers(id),
    categoryId INTEGER NOT NULL REFERENCES Categories(id),
    type TEXT NOT NULL CHECK (type IN ('TEXTBOOK', 'ELECTRONICS', 'FURNITURE')),
    title TEXT NOT NULL CHECK (length(trim(title)) > 0),
    priceMinor INTEGER NOT NULL

        CHECK (typeof(priceMinor) = 'integer' AND priceMinor > 0),
    currency TEXT NOT NULL

        CHECK (length(currency) = 3 AND currency NOT GLOB '*[^A-Z]*'),
    status TEXT NOT NULL CHECK (status IN ('ACTIVE', 'SOLD', 'REMOVED')),
    created_at TEXT CHECK (created_at IS NULL OR length(trim(created_at)) > 0)
);
```

Reviews

```
CREATE TABLE Reviews (
    id INTEGER PRIMARY KEY,
    listingId INTEGER NOT NULL REFERENCES Listings(id),
    reviewerId INTEGER NOT NULL REFERENCES Members(id),
    rating INTEGER NOT NULL

        CHECK (typeof(rating) = 'integer' AND rating BETWEEN 1 AND 5),
    comment TEXT
);
```

BuyerInterests

```
CREATE TABLE BuyerInterests (
    id INTEGER PRIMARY KEY,
    listingId INTEGER NOT NULL REFERENCES Listings(id),
    buyerId INTEGER NOT NULL REFERENCES Members(id),
    joiningOrder INTEGER NOT NULL

        CHECK (typeof(joiningOrder) = 'integer' AND joiningOrder > 0),
    UNIQUE (listingId, buyerId),
    UNIQUE (listingId, joiningOrder)
);
```

HistoricalSales

```
CREATE TABLE HistoricalSales (
    id INTEGER PRIMARY KEY,
    categoryId INTEGER NOT NULL REFERENCES Categories(id),
    sourceRecordId TEXT NOT NULL UNIQUE
        CHECK (length(trim(sourceRecordId)) > 0),
    priceMinor INTEGER NOT NULL

        CHECK (typeof(priceMinor) = 'integer' AND priceMinor > 0),
    currency TEXT NOT NULL

        CHECK (length(currency) = 3 AND currency NOT GLOB '*[^A-Z]*'),
    soldAt TEXT NOT NULL CHECK (length(trim(soldAt)) > 0)
);
```

TextbookDetails

```
CREATE TABLE TextbookDetails (
    listingId INTEGER PRIMARY KEY REFERENCES Listings(id) ON DELETE CASCADE,
    missingPages INTEGER NOT NULL

        CHECK (typeof(missingPages) = 'integer' AND missingPages >= 0),
    markingLevel TEXT NOT NULL CHECK (markingLevel IN ('NONE', 'LIGHT', 'HEAVY')),
    coverDamaged INTEGER NOT NULL

        CHECK (typeof(coverDamaged) = 'integer' AND coverDamaged IN
        (0, 1))
);
```

ElectronicsDetails

```
CREATE TABLE ElectronicsDetails (
    listingId INTEGER PRIMARY KEY REFERENCES Listings(id) ON DELETE CASCADE,
    powersOn INTEGER NOT NULL

        CHECK (typeof(powersOn) = 'integer' AND powersOn IN
        (0, 1)),
    hasFunctionalDefect INTEGER NOT NULL

        CHECK (typeof(hasFunctionalDefect) = 'integer' AND hasFunctionalDefect IN
        (0, 1)),
    hasCosmeticDamage INTEGER NOT NULL

        CHECK (typeof(hasCosmeticDamage) = 'integer' AND hasCosmeticDamage IN
        (0, 1))
);
```

FurnitureDetails

```
CREATE TABLE FurnitureDetails (
    listingId INTEGER PRIMARY KEY REFERENCES Listings(id) ON DELETE CASCADE,
    structurallyStable INTEGER NOT NULL

        CHECK (typeof(structurallyStable) = 'integer' AND structurallyStable IN
        (0, 1)),
    hasStructuralDamage INTEGER NOT NULL

        CHECK (typeof(hasStructuralDamage) = 'integer' AND hasStructuralDamage IN
        (0, 1)),
    surfaceWear TEXT NOT NULL CHECK (surfaceWear IN ('NONE', 'LIGHT', 'HEAVY'))
);
```

5. API Integration Design

Catalog request and response

The public endpoint is GET https://fakestoreapi.com/products. It returns a JSON array and needs no API key. Suggest category / price makes a fresh request.

Example response shape (illustrative values, not a captured live response):

```
[
  {
    "id": 1, "title": "Example monitor", "price": 120.50,
    "category": "electronics", "description": "Product details",
    "image": "https://example.com/product.png",
    "rating": {"rate": 4.2, "count": 25}
  }
]
```

ProductCatalogClient sends the request with a 15-second timeout, checks HTTP 200 and records request time/status. ProductCatalogParser validates one complete array with Gson: id must be a positive whole number, price a positive number, and title/category nonblank text. Trailing data, invalid shapes or missing required fields fail. CatalogProduct carries id, title, category and BigDecimal price. CatalogResponse carries products and request information. Unused description, image and rating fields are ignored.

Mapping and listing-form use

CatalogMapper.matches() selects compatible electronics products sharing title words with the listing. It ignores short/common words, ranks by shared-word count, breaks ties by product ID and returns up to three references. CatalogService prepares titles and minimum/maximum reference price for ListingController. The member chooses the campus category and final PHP price manually.

The API does not state currency, so prices are labeled currency unspecified. Unsupported clothing/jewelry categories are not converted into campus listing types. Textbooks, furniture and unmatched queries retain manual entry. Posting inputs are disabled during a request, preventing a stale response from being applied to changed inputs.

Errors and API verification

HTTP failure, timeout or invalid JSON produces a retry/manual-entry message through task error state. It does not save a listing or erase entered values. Parser/client tests use local HTTP servers. Capturing a live response and demonstrating a network toggle are separate checks; an illustrative example is not live evidence.

6. Concurrency Design

Worker and controller state

App creates one named ExecutorService, marketplace-database, shared by the controllers. Views create no threads and call no DAOs or HTTP clients. FxTaskRunner.run() wraps slow work in JavaFX Task. Success/failure handlers update controller state on the JavaFX thread; views bind that state.

Initialization, registration, login, reset, listing actions, catalog requests, JSON import and analytics run on this worker. Controllers capture inputs and active member before starting. UserSession's generation number lets listing/analysis results reject a signed-out or changed session.

TaskRunner is injectable: controller tests use a deterministic runner; the app uses FxTaskRunner. Services convert data-access failures into AppException, so views do not handle JDBC exceptions.

Background JSON import

SalesHistoryImporter streams a UTF-8 JSON array one record at a time. Each record contains sourceRecordId, categoryPath, price, currency and soldAt. It validates required text, an unambiguous existing category path, positive exact minor-unit price, uppercase currency and an offset timestamp.

HistoryDao supplies a prepared insert. One transaction commits valid new rows after the file ends successfully. Invalid field values and duplicate source IDs are skipped and counted. Broken JSON, database failure or cancellation rolls back that import's new rows and preserves previous history.

AnalyticsController dispatches progress to JavaFX after the first row and every 1,000 rows. An indeterminate spinner/bar shows real import activity without inventing a percentage. Cancel sets a flag checked between records. Success shows saved/invalid/duplicate totals, up to five validation messages and the latest 20 rows. Preview failure after commit still reports that the import was saved.

Automatic updates between windows

FxRefreshSchedule supplies a three-second Timeline. RefreshController.tick() polls only while signed in, on Marketplace or Analytics, with no active work or open dialog. Listing results must still match the session, category, query and My listings filter. Unchanged values preserve rows and selection; equal analytics snapshots avoid redraw.

The timer stops on logout and close. Failed reads show a message and allow retry. Windows using one SQLite file see another connection's post/withdrawal on a successful check; separate laptop databases are not synchronized. Details/reviews/waitlists reload through their corresponding actions.

Responsiveness and shutdown

Work buttons and posting inputs are disabled while busy. Loaded views can scroll and navigation remains available. Cancel import stays available during import. App.stop() stops refresh, requests cancellation and shuts down the worker. Every database operation closes its resources. Polling very large databases still needs performance measurement.

7. Data Mining & Visualization Design

Weekly sales frequency in Java

The analysis asks how many imported sales occurred each week and which observed weeks were busiest. HistoryDao.soldDates() reads HistoricalSales.soldAt. SalesAnalysis.weeklyCounts() converts timestamps to UTC, finds the previous-or-same Monday and increments a TreeMap count. The map sorts the weeks. WeeklySales records carry the results, and summary() finds the total, largest count and all tied busiest weeks.

This is Java aggregation, not SQL AVG/COUNT. Counts include all currencies because they count sales, not money. Duplicate source IDs are not inserted twice. Weeks without sales are not plotted; empty history has a no-history message. UTC grouping also handles offset and year boundaries consistently.

The six sample sales produce counts 3, 2 and 1 for weeks beginning 19 October, 26 October and 2 November 2026. The first week is busiest; existing history can change the totals.

Category averages

Table 9: Independent category calculations

Result	Calculation and data
Average listing price	ACTIVE Listings in one selected currency: AVG(priceMinor) / 100.0 and COUNT(*), grouped by category.
Average review rating	Individual Reviews on ACTIVE Listings: AVG(rating) and COUNT(*), grouped by category. Tied best-rated categories show review counts.

AnalyticsDao runs separate queries so reviews do not multiply listing prices. Currencies remain separate. Historical sales and external API values do not enter these means. Categories without reviews are unrated, not zero stars.

Charts and why they fit

Three discrete bar charts compare groups using separated colored bars, light dashed grid lines and exact-value tooltips:

- Weekly frequency: X-axis Week starting Monday (UTC); Y-axis Sales.
- Listing means: X-axis Category; Y-axis Price in the selected currency, initially PHP.
- Review means: X-axis Category; Y-axis Rating on a 0-5 scale.

AnalyticsService returns an AnalyticsSnapshot; AnalyticsController publishes it and ChartsView only renders it. Titles, axes, busiest-week/tie summaries and no-data messages explain the result. Manual refresh and idle polling read current data.

Checking that the analysis changes

HistoryAnalysisTest imports, deletes and reimports saved rows and checks changed counts. SalesAnalysisTest checks ordering, UTC/year boundaries, ties, empty data and immutable results. Current listing-price means remain independent of historical frequencies.

Appendix - References and reproduction

design-document.md is the editable master. Its Mermaid blocks define the hierarchy and database diagrams; assets/ contains the editable draw.io sketch and PNG. From the repository root, `python tools/export_design.py` rebuilds the PDF using Python, ReportLab and Calibri. Set CALIBRI_DIR for another font folder.

Use JDK 21 and keep the version enforcer enabled. From program/, run `java -version`, `./mvnw.cmd -version` and `./mvnw.cmd clean verify`. Both version checks must show Java 21. The existing Eclipse and Antigravity Run profiles remain available.

Account setup and recovery

Initial local users are Person A / person_a, Person B / person_b and Person C / person_c. A new setup displays random passwords and recovery codes once in a local dialog. Save them privately. Existing accounts retain their credentials. A member can also register a new account.

After registration or reset, save the displayed recovery code. A reset consumes the previous code and supplies a replacement. There are no published starter secrets or email-delivery promises.

Verification

JDK 21 Maven verification on 13 November 2026 passed 119 JUnit tests in eleven classes: zero failures/errors/skips, enforcer enabled. Temporary databases and local HTTP servers are used. Added cases cover migration/rollback, creation dates, sold-state preservation, missing rows, two-thread waitlist joins and fast filter changes; earlier account, model, analysis and import checks remain.

The 40 separate application checks passed: account/review flows, posting, sold/remove confirmations, archive filters, import/charts, cancellation rollback, external refresh, missing electronics/furniture details, owner repair and resizing. They use temporary data and are separate from the JUnit count.

A new live public API check was unavailable because the checking session's proxy refused the connection. Repeat the response capture/network-toggle check on the normal connection; local server tests are separate evidence.

Limitations and requirement mapping

Accounts and records are local to one SQLite file. There is no cloud synchronization, email recovery, payment processing or proof of completed purchases. Recent-view stacks last for the service session. A personal dashboard and waitlist leave/position screens are not implemented. Very large-dataset polling is not benchmarked.

KNOWN_ISSUES.md and REQUIREMENT_MAP.md in the repository root list limits and map checklist requirements to classes, methods and tests. Splitting the code does not add database fields.

- Team repository
- Project proposal
- FakeStoreAPI documentation
- Project Design Document Template
- Project Design Document Checklist

Program process map

All types are in edu.uc.campusmarket or the listed subpackage.

Table 10: Implemented process flow

Process	Screen and controller	Service and storage result
Startup	App.start(); AccountController.initialize()	MemberService initializes Database/AccountSetup and returns newly issued credentials once.
Register	RegistrationView; AccountController.register()	MemberDao saves a unique account and returns a recovery code.
Login / reset	LoginView / PasswordResetView; AccountController.login()/reset()	MemberDao verifies or replaces hashes; UserSession holds active identity.
Cards / search	ListingTableView / ListingCards; ListingController.refresh()	ListingService and MarketplaceService search available listings and category descendants.
Post / sold / remove	ListingFormView; ListingController. post()/markSold()/withdraw()	ListingFactory creates a subtype; ListingDao records created_at or rechecks owner before changing ACTIVE to SOLD/REMOVED.
History / waitlist	ListingController.viewDetails(); WaitlistController.join()	MarketplaceService uses LIFO/FIFO; InterestDao allocates and inserts joining order atomically.
Review	ReviewsView; ReviewController.submit()	ReviewDao checks owner, status, interest, rating and duplicate before saving.
Restore evidence	ListingTableView; ListingController. repairable()/restore()	ListingDao checks owner, type and missing details and saves observed evidence.
API reference	ListingFormView; ListingController.suggest()	CatalogService -> client/parser -> DTOs -> CatalogMapper prepares references.
Import	ImportView; AnalyticsController.im portFile()/cancelImport()	Importer validates rows; HistoryDao supplies the insert; cancellation/failure rolls back.
Analysis / charts	ChartsView; AnalyticsController.refresh()	AnalyticsService combines DAO means and Java weekly counts in AnalyticsSnapshot.
Refresh	RefreshController.tick(); FxRefreshSchedule	Polls active idle pages; session/filter checks reject stale results.
Sign out / close	AccountController.logout(); App.stop()	Clears session, stops timer, requests cancellation and shuts down the worker.

Complete Java type inventory

The current source contains 69 top-level Java types. Every one is listed below; the database still contains ten tables. A DTO, controller or view is not an additional database table.

Table 11: Java types by package

Package	Types
base	App
model	AnalyticsSnapshot, Category, CategorySummary, ConditionGrade, Electronics, Furniture, ImportResult, InitialAccount, Listing, ListingForRepair, Member, Registration, Review, Seller, Textbook, WearLevel, WeeklySales
dto	CatalogProduct, CatalogResponse
api	ProductCatalogClient, ProductCatalogParser
dao	AnalyticsDao, CategoryDao, HistoryDao, InterestDao, ListingDao, MemberDao, ReviewDao, SellerDao
persistence	AccountSetup, Database
service	AnalyticsService, AppException, CatalogMapper, CatalogService, CredentialSecurity, ListingFactory, ListingService, MarketplaceService, MemberService, ReviewService, SalesAnalysis, SalesHistoryImporter, ServiceCall, WaitlistService
controller	AccountController, AnalyticsController, FxRefreshSchedule, FxTaskRunner, ListingController, RefreshController, RefreshSchedule, ReviewController, TaskRunner, UserSession, WaitlistController
ui	ChartsView, ImportView, ItemArtwork, ListingCards, ListingFormView, ListingTableView, LoginView, MarketplaceView, PasswordResetView, RegistrationView, ReviewsView, UiTools

ServiceCall.Query is a nested SQL-operation interface. Anonymous Tasks execute background actions. Listing.incomplete() creates an anonymous read-only fallback for unavailable condition evidence; it is not a new posted item type or table.